Northstar Data Model

Oct 22, 2026 · @Abhishek

1. Conventions

The schema runs on Supabase Postgres (Mumbai), managed by Flyway migrations in `db/migrations/`; every rule below applies to every table unless the table says otherwise.

Topic	Rule
Primary keys	`id uuid primary key default gen_random_uuid()`
Tenant scope	Every business-owned table has `tenant_id uuid not null references tenants(id)`, including join and history tables
Timestamps	`created_at timestamptz not null default now()`, `updated_at timestamptz` (trigger-maintained); stored in UTC, shown in the tenant timezone
Money	`bigint` in paise (₹1 = 100) with `currency char(3) default 'INR'`; never floats
Tax rates	`numeric(5,2)` percent (e.g. 18.00)
Phones	E.164 text (`+919876543210`) in `normalized_phone`
Enumerations	`text` + `check` constraint listing allowed values (easier to migrate than Postgres enum types)
Flexible fields	`jsonb` only for business-specific extras (`requirements` , `config`); anything queried or joined is a typed column
Soft delete	`deleted_at timestamptz` on contacts, leads, catalogue items, users; financial records are never deleted

Topic	Rule
Optimistic locking	`version int not null default 1` on leads, quotes, appointments; updates check and increment it
Append-only	`timeline_events`, `audit_log`, `webhook_events`, `lead_score_history`, `payments` status history: insert only
Secrets	Never in tables readable by tenants; credentials stored encrypted, referenced by id
Naming	snake_case, plural table names, `<table>_id` foreign keys

2. At a glance

About 60 tables in nine groups; the lead sits at the centre and every group links to it.

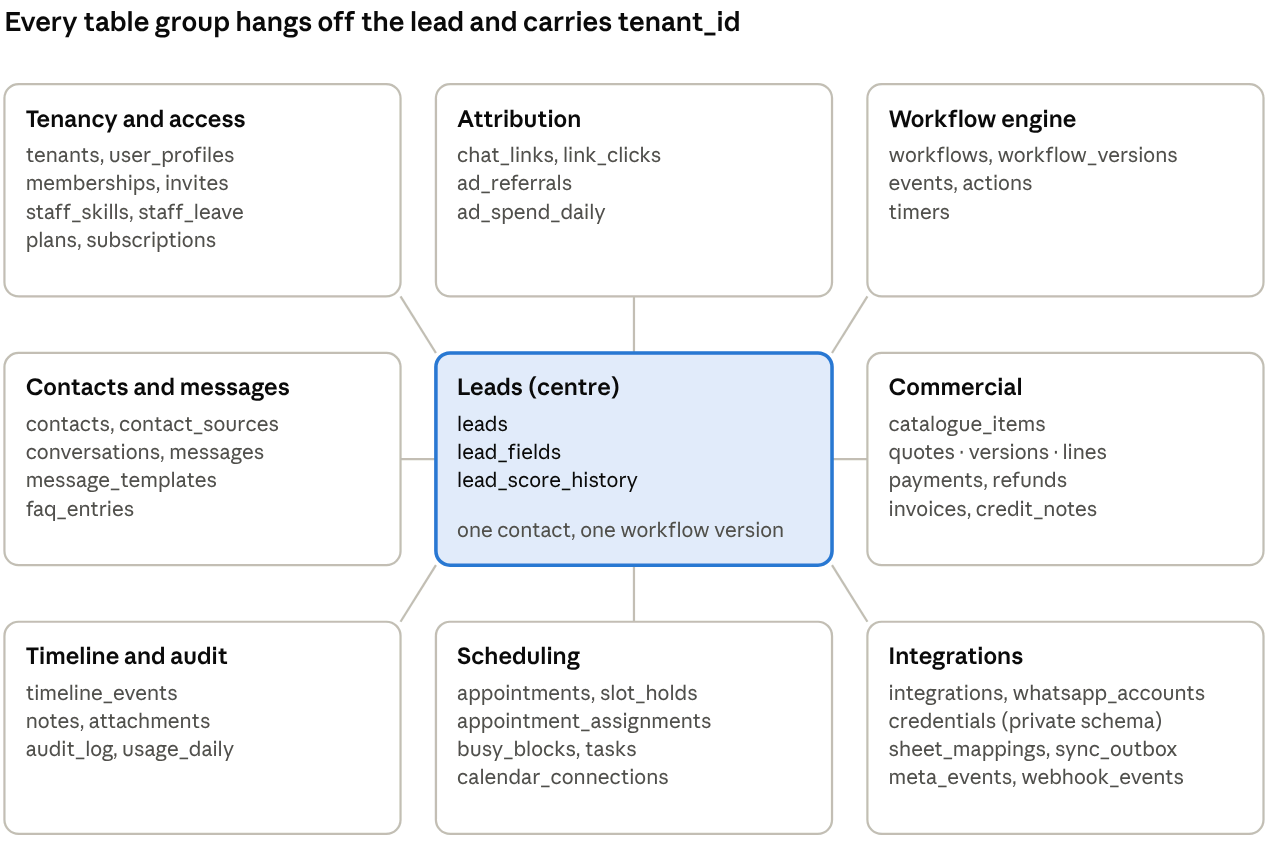


table groups · 9 groups around the lead

Tenancy links to everything through tenant_id; webhook_events is the one table whose tenant is resolved after arrival.

Full entity-relationship diagram

All 59 tables and their foreign keys. Every table also carries `tenant_id`; tenant links are drawn only for tables with no other parent, to keep the diagram readable.

```
erDiagram
  %% Tenancy and platform billing
  tenants ||--o{ memberships : has
  user_profiles ||--o{ memberships : "belongs via"
  tenants ||--o{ invites : sends
  memberships ||--o{ staff_skills : has
  memberships ||--o{ staff_leave : takes
  plans ||--o{ subscriptions : "priced by"
  tenants ||--o{ subscriptions : has
  subscriptions ||--o{ subscription_invoices : bills
  tenants ||--o{ usage_alerts : receives

  %% Contacts and messages
  tenants ||--o{ contacts : owns
  contacts ||--o{ contact_sources : "arrived from"
  contacts ||--o{ conversations : has
  conversations ||--o{ messages : contains
  tenants ||--o{ message_templates : owns
  message_templates ||--o{ messages : "used by"
  tenants ||--o{ faq_entries : owns

  %% Attribution
  tenants ||--o{ chat_links : creates
  chat_links ||--o{ link_clicks : logs
  contacts ||--o{ ad_referrals : "came via"
  tenants ||--o{ ad_spend_daily : tracks
  chat_links ||--o{ contact_sources : tags
  ad_referrals ||--o{ contact_sources : tags

  %% Workflow and leads
  tenants ||--o{ workflows : configures
  workflows ||--o{ workflow_versions : publishes
  contacts ||--o{ leads : "has"
  conversations ||--o{ leads : "talks in"
  workflow_versions ||--o{ leads : runs
```

```
ad_referrals |o--o{ leads : sources
chat_links |o--o{ leads : sources
link_clicks }o--o| leads : "matched to"
memberships |o--o{ leads : "assigned to"
leads ||--o{ lead_fields : stores
messages |o--o{ lead_fields : "extracted from"
leads ||--o{ lead_score_history : scores
leads ||--o{ events : receives
leads ||--o{ actions : runs
leads ||--o{ timers : schedules

%% Commercial
tenants ||--o{ catalogue_items : sells
leads ||--o{ quotes : gets
quotes ||--o{ quote_versions : versions
quote_versions ||--o{ quote_lines : lists
catalogue_items |o--o{ quote_lines : "copied into"
leads ||--o{ payments : pays
quote_versions ||--o{ payments : "accepted as"
payments ||--o{ invoices : "invoiced by"
tenants ||--o{ invoice_sequences : numbers
tenants ||--o{ invoice_templates : designs
invoice_templates |o--o{ invoices : formats
invoices ||--o{ invoice_lines : lists
invoices ||--o{ credit_notes : "cancelled by"
payments ||--o{ refunds : refunds
credit_notes |o--o| refunds : documents
tenants ||--o{ cancellation_policies : sets

%% Scheduling
memberships ||--o{ calendar_connections : connects
memberships ||--o{ busy_blocks : "is busy"
leads ||--o{ slot_holds : holds
memberships ||--o{ slot_holds : "held for"
payments |o--o| slot_holds : secures
leads ||--o{ appointments : books
memberships ||--o{ appointments : "works"
appointments ||--o{ appointment_assignments : "reassigned via"
leads ||--o{ tasks : has
appointments |o--o{ tasks : includes
memberships ||--o{ tasks : owns

%% Integrations
```

```
  tenants ||--o{ integrations : connects
  integration_credentials ||--o{ integrations : authorises
  integration_credentials ||--o{ calendar_connections : authorises
  tenants ||--o{ whatsapp_accounts : owns
  tenants ||--o{ sheet_mappings : maps
  tenants ||--o{ sync_outbox : queues
  tenants ||--o{ sheet_conflicts : reviews
  leads ||--o{ meta_events : reports
  tenants |o--o{ webhook_events : "resolved to"

  %% Timeline and audit
  leads ||--o{ timeline_events : records
  leads ||--o{ notes : has
  leads ||--o{ attachments : stores
  tenants ||--o{ audit_log : audits
  tenants ||--o{ usage_daily : meters
```

Notation: `||--o{` one to many, `|o--o{` optional one to many, `|o--o|` optional one to one, `}o--o|` many to optional one.

3. Tenancy and access

A tenant is one client business; users come from Supabase Auth and get access to a tenant only through a membership row.

Table	Columns (type)	Constraints / notes
`tenants`	id, name, slug (text), industry (text), timezone (text, default 'Asia/Kolkata'), locale (text), brand_tone (text), legal_name (text), gstin (text), state_code (char(2)), billing_address (jsonb), approval_mode (bool, default true), quiet_hours (jsonb), status (text), plan_id (uuid), activated_at, created_at, updated_at	slug unique; status in ('onboarding','active','paused','suspended','closed'); not active until paid or trial override
`user_profiles`	id (= auth.users.id), full_name, phone, avatar_path, is_platform_admin (bool), created_at, deleted_at	One row per Supabase Auth user; is_platform_admin only settable by Northstar
`memberships`	id, tenant_id, user_id, role (text), status (text), whatsapp_phone (text), created_at, updated_at	unique (tenant_id, user_id); role in ('admin','staff'); status in ('invited','active','deactivated')

Table	Columns (type)	Constraints / notes
`invites`	id, tenant_id, email (citext), role, token_hash (text), invited_by, expires_at, accepted_at, revoked_at	Token stored hashed; expires 7 days after creation
`staff_skills`	id, tenant_id, membership_id, skill (text)	unique (membership_id, skill); used by assignment rules
`staff_leave`	id, tenant_id, membership_id, starts_on (date), ends_on (date), reason (text), created_by	ends_on ⩾ starts_on; overlapping appointments flagged on insert

4. Contacts, conversations and messages

A contact is one phone number per tenant; it can have many leads over time but one ongoing WhatsApp conversation.

Table	Columns (type)	Constraints / notes
`contacts`	id, tenant_id, normalized_phone, name, email, is_repeat_customer (bool), consent_state (text), opted_in_at, opt_out_at, attributes (jsonb), created_at, updated_at, deleted_at	unique (tenant_id, normalized_phone); consent_state in ('unknown','opted_in','opted_out')
`contact_sources`	id, tenant_id, contact_id, source (text), chat_link_id, ad_referral_id, first_seen_at	One row per distinct source a contact arrived from

Table	Columns (type)	Constraints / notes
`conversations`	id, tenant_id, contact_id, channel (text), whatsapp_phone_number_id (text), status (text), ai_paused (bool), assigned_membership_id, last_inbound_at, window_expires_at, needs_human (bool), created_at, updated_at	unique (tenant_id, contact_id, channel); window_expires_at = last_inbound_at + 24 h
`messages`	id, tenant_id, conversation_id, direction (text), sender_type (text), sender_membership_id, provider_message_id (text), type (text), body (text), media_path, template_id, status (text), error_code, sent_at, delivered_at, read_at, created_at	unique (tenant_id, provider_message_id); direction in ('in','out'); sender_type in ('contact','ai','staff','system'); status in ('queued','sent','delivered','read','failed')
`message_templates`	id, tenant_id, name, language, category (text), body, variables (jsonb), provider_template_id, status (text), approved_at	unique (tenant_id, name, language); status in ('draft','submitted','approved','rejected','paused'); category as returned by Meta
`faq_entries`	id, tenant_id, question, answer, is_active, updated_by, updated_at	Only source the AI may answer from

5. Attribution

Every lead points to exactly one origin: a Meta ad referral, a chat-link click, or direct.

Table	Columns (type)	Constraints / notes
`chat_links`	id, tenant_id, code (text), name, source (text), utm (jsonb), prefill_text, qr_path, is_active, created_by, created_at	code unique platform-wide (used in `northstar.com/c/<code>`); source in ('website','email','instagram','facebook','other')

Table	Columns (type)	Constraints / notes
`link_clicks`	id, tenant_id, chat_link_id, ref_code (text), clicked_at, page_url, utm (jsonb), user_agent_hash, matched_lead_id	ref_code unique; matched when the pre-filled message arrives; raw IP not stored
`ad_referrals`	id, tenant_id, contact_id, ctwa_click_id (text), source_id (ad id), source_type, source_url, headline, campaign_id, campaign_name, adset_id, ad_name, received_at	Filled from the WhatsApp referral payload; campaign names enriched from Marketing API
`ad_spend_daily`	id, tenant_id, date, campaign_id, adset_id, ad_id, spend_paise, impressions, clicks, pulled_at	unique (tenant_id, date, ad_id); for cost-per-qualified-lead reports

6. Workflow and leads

The lead is the central record: it belongs to a contact, runs on one workflow version, and every other business table hangs off it.

Table	Columns (type)	Constraints / notes
`workflows`	id, tenant_id, name, template (text), current_version_id, draft_config (jsonb), created_at, updated_at	One active workflow per tenant in MVP
`workflow_versions`	id, tenant_id, workflow_id, version (int), config (jsonb), published_by, published_at	unique (workflow_id, version); immutable after insert (no update grant)

Table	Columns (type)	Constraints / notes
`leads`	id, tenant_id, contact_id, conversation_id, workflow_version_id, stage (text), previous_stage (text), stage_entered_at, origin (text), ad_referral_id, chat_link_id, service (text), event_date (date), event_time (time), location (text), location_geo (point), budget_paise (bigint), requirements (jsonb), score (smallint), band (text), score_reasons (jsonb), junk_rule_id (text), lost_reason (text), assigned_membership_id, amount_due_paise (bigint), version (int), created_at, updated_at, closed_at, deleted_at	stage must exist in the version config; band in ('hot','warm','cold','junk'); origin in ('meta_ad','chat_link','direct','manual'); score 0–100
`lead_fields`	id, tenant_id, lead_id, field (text), value (jsonb), confidence (numeric(3,2)), source_message_id, set_by (text), created_at	Every extracted/edited field value with provenance; latest per field = current
`lead_score_history`	id, tenant_id, lead_id, score, band, reasons (jsonb), model, prompt_version, created_at	Append-only
`events`	id, tenant_id, lead_id, type (text), dedupe_key (text), payload (jsonb), received_at, processed_at	unique (tenant_id, dedupe_key); engine input log
`actions`	id, tenant_id, lead_id, type (text), status (text), idempotency_key (text), mode (text), payload (jsonb), result (jsonb), attempts (int), next_attempt_at, last_error, approver_membership_id, approved_at, stage_entry_id, created_at, updated_at	unique (tenant_id, idempotency_key); status in ('pending','processing','succeeded','failed','needs_review','skipped','cancelled'); mode in ('approval','automatic')

Table	Columns (type)	Constraints / notes
`timers`	id, tenant_id, lead_id, type (text), fire_at, job_id (text), status (text), created_at	Mirror of scheduled JobRunr jobs so the engine can cancel them; status in ('scheduled','fired','cancelled')

JobRunr creates its own `jobrunr_*` tables in a separate schema; they hold no tenant data beyond IDs in job arguments.

7. Commercial

Quotes are versioned, payments form a ledger keyed by Razorpay IDs, and invoices are numbered per financial year and never deleted.

Table	Columns (type)	Constraints / notes
`catalogue_items`	id, tenant_id, name, description, unit_price_paise, hsn_sac (text), tax_rate (numeric(5,2)), is_active, created_at, updated_at, deleted_at	Changing a price never changes sent quotes
`quotes`	id, tenant_id, lead_id, current_version_id, status (text), created_at	status in ('open','accepted','closed')
`quote_versions`	id, tenant_id, quote_id, version (int), subtotal_paise, discount_paise, tax_paise, total_paise, valid_until (date), status (text), discount_approved_by, pdf_path, sent_at, accepted_at, created_by, created_at	unique (quote_id, version); status in ('draft','needs_review','sent','accepted','changes_requested','superseded','expired'); only latest sent version acceptable

Table	Columns (type)	Constraints / notes
`quote_lines`	id, tenant_id, quote_version_id, catalogue_item_id, description, qty (numeric), unit_price_paise, hsn_sac, tax_rate, line_total_paise	Copied values, not references, so history stays exact
`payments`	id, tenant_id, lead_id, quote_version_id, kind (text), amount_paise, status (text), razorpay_link_id, razorpay_link_url, razorpay_payment_id, link_expires_at, paid_at, failure_reason, created_at, updated_at	unique (tenant_id, razorpay_payment_id) where not null; kind in ('full','advance','partial','balance','fee'); status in ('pending','paid','failed','expired','refunded','partially_refunded','refund_due')
`invoice_sequences`	id, tenant_id, financial_year (text, e.g. '2026-27'), prefix, next_number (int)	unique (tenant_id, financial_year); incremented in the same transaction as invoice insert, so no gaps
`invoices`	id, tenant_id, lead_id, payment_id, number (text), financial_year, issue_date, place_of_supply (char(2)), supplier_state (char(2)), subtotal_paise, cgst_paise, sgst_paise, igst_paise, total_paise, status (text), template_id, pdf_path, sent_at, cancelled_at, created_at	unique (tenant_id, number); status in ('draft','issued','partially_paid','paid','overdue','cancelled'); no delete grant
`invoice_lines`	id, tenant_id, invoice_id, description, hsn_sac, qty, unit_price_paise, tax_rate, taxable_paise, tax_paise	Copied values

Table	Columns (type)	Constraints / notes
`credit_notes`	id, tenant_id, invoice_id, number, reason, amount_paise, tax_paise, issue_date, pdf_path, created_by	unique (tenant_id, number)
`refunds`	id, tenant_id, payment_id, amount_paise, policy_snapshot (jsonb), status (text), requested_by, approved_by, approved_at, razorpay_refund_id, processed_at, credit_note_id	status in ('requested','approved','rejected','processing','processed','failed')
`invoice_templates`	id, tenant_id, name, docx_path, field_mapping (jsonb), is_default, created_by, created_at	One default per tenant (partial unique index)
`cancellation_policies`	id, tenant_id, rules (jsonb), is_active, updated_at	e.g. hours-before thresholds → refund %

8. Scheduling

The platform's appointments table is the master schedule; Google Calendar event IDs are stored alongside so every change can be mirrored.

Table	Columns (type)	Constraints / notes
`calendar_connections`	id, tenant_id, membership_id (null = business calendar), google_calendar_id, credential_id, status (text), last_synced_at	status in ('connected','error','disconnected')
`busy_blocks`	id, tenant_id, membership_id, starts_at, ends_at, source (text), google_event_id, is_private (bool), synced_at	Free/busy cache from Google; private events stored as busy only, no title
`slot_holds`	id, tenant_id, lead_id, membership_id, starts_at, ends_at, payment_id, expires_at, status (text)	status in ('held','converted','released'); exclusion constraint prevents overlapping active holds per staff

Table	Columns (type)	Constraints / notes
`appointments`	id, tenant_id, lead_id, kind (text), starts_at, ends_at, location, assigned_membership_id, status (text), google_event_ids (jsonb), version (int), created_at, updated_at, cancelled_at	kind in ('call','site_visit','job','meeting'); status in ('scheduled','in_progress','done','cancelled','rescheduled'); exclusion constraint on (assigned_membership_id, tstzrange) for active rows
`appointment_assignments`	id, tenant_id, appointment_id, from_membership_id, to_membership_id, reason (text), changed_by, changed_at	Append-only reassignment history
`tasks`	id, tenant_id, lead_id, appointment_id, title, owner_membership_id, due_at, status (text), completed_at, created_by, created_at	status in ('open','done','overdue','cancelled')

Staff leave lives in `staff_leave` (section 3).

9. Integrations, sync and webhooks

Credentials sit in a server-only table that tenants can never read; everything leaving the platform for Sheets or Meta goes through an outbox.

Table	Columns (type)	Constraints / notes
`integrations`	id, tenant_id, provider (text), status (text), account_label, scopes (text[]), credential_id, last_success_at, last_error, last_error_at, created_at, updated_at	unique (tenant_id, provider) except calendars; provider in ('whatsapp','razorpay','google','meta_ads','meta_capi')
`integration_credentials`	id, tenant_id, provider, encrypted_secret (bytea), key_id (text), expires_at, rotated_at	**Private schema**, no RLS access for app users; read only by the backend service role; encrypted with a managed key
`whatsapp_accounts`	id, tenant_id, waba_id, phone_number_id, display_phone, display_name, quality_rating, messaging_limit, verified_at	unique (phone_number_id) platform-wide; routes inbound webhooks to the tenant

Table	Columns (type)	Constraints / notes
`sheet_mappings`	id, tenant_id, spreadsheet_id, tab (text), entity (text), field_map (jsonb), inbound_fields (text[]), last_synced_at, status	entity in ('leads','payments')
`sync_outbox`	id, tenant_id, target (text), entity, entity_id, operation (text), payload (jsonb), status (text), attempts, next_attempt_at, last_error, created_at	target in ('sheets','meta_capi'); processed in batches by the worker
`sheet_conflicts`	id, tenant_id, entity_id, field, sheet_value, platform_value, detected_at, resolved_by, resolved_at	Review list for protected-field edits
`meta_events`	id, tenant_id, lead_id, event_name (text), event_id (text), value_paise, ctwa_click_id, sent_at, response (jsonb), status	unique (tenant_id, event_id) so Meta can deduplicate
`webhook_events`	id, provider, provider_event_id, tenant_id (nullable until resolved), signature_valid (bool), payload (jsonb), received_at, processed_at, error	unique (provider, provider_event_id); append-only; tenant resolved from phone_number_id / account

10. Timeline, audit and usage

The customer timeline is one append-only table that every module writes to, so the dashboard shows a lead's whole history from a single query.

Table	Columns (type)	Constraints / notes
`timeline_events`	id, tenant_id, lead_id, contact_id, type (text), actor_type (text), actor_membership_id, ref_table (text), ref_id (uuid), summary (text), data (jsonb), occurred_at	Append-only; actor_type in ('ai','system','staff','contact'); types include stage_changed, message, quote_sent, payment_paid, invoice_issued, appointment_set, reassigned, approval, call_logged, note, delivery
`notes`	id, tenant_id, lead_id, body, author_membership_id, created_at	Staff notes and logged calls (with `kind` = 'note' or 'call')
`attachments`	id, tenant_id, lead_id, path, mime_type, size_bytes, kind (text), uploaded_by, created_at	Delivery evidence, documents; files in Supabase Storage under `tenant_id/` prefix
`audit_log`	id, tenant_id, actor_user_id, action (text), target_table, target_id, metadata (jsonb), ip_hash, created_at	Settings, roles, deletes, exports, approvals, force moves; append-only
`usage_daily`	id, tenant_id, date, messages_in, messages_out, templates_sent, ai_calls, ai_input_tokens, ai_output_tokens, storage_bytes, pdfs_generated	unique (tenant_id, date); feeds plan limits and the admin panel

11. Platform billing

These tables track Northstar's own subscription with each business, separate from the business's payments from its customers.

Table	Columns (type)	Constraints / notes
`plans`	id, code (text), name, price_paise, billing_period (text), limits (jsonb: messages, ai_calls, staff_seats, storage_gb), is_active	Platform-wide, no tenant_id; readable by all, writable by Northstar admin only

Table	Columns (type)	Constraints / notes
`subscriptions`	id, tenant_id, plan_id, status (text), current_period_start, current_period_end, trial_ends_at, grace_ends_at, setup_fee_paise, created_at, updated_at	status in ('trial','active','past_due','grace','read_only','cancelled')
`subscription_invoices`	id, tenant_id, subscription_id, number, period_start, period_end, amount_paise, gst_paise, status, razorpay_payment_id, pdf_path, issued_at, paid_at	Northstar's invoices to the business
`usage_alerts`	id, tenant_id, metric (text), threshold_pct (int), notified_at	Alerts at 70%, 90%, 100% of plan limits

12. Row Level Security and tenant isolation

The React app never queries tables directly; Spring Boot connects as a restricted role and sets the tenant on every transaction, and RLS rejects any row from another tenant even if application code has a bug.

Roles

Role	Used by	Can bypass RLS
`app_user`	Spring Boot API and worker for all tenant work	No
`app_admin`	Northstar admin panel queries across tenants (separate code path, audited)	Yes, read-mostly
`migrator`	Flyway only	Yes
Supabase `service_role` key	Never in the app runtime; kept for emergencies	Yes

Per-transaction context (set by a Spring interceptor from the verified Supabase token and membership lookup; workers set it from the job's tenant_id):

```
select set_config('app.tenant_id', :tenant_id, true);   -- true = local to
this transaction
select set_config('app.user_id',   :user_id,   true);
```

Policy applied to every tenant table (generated by a migration helper so none is missed):

```
alter table leads enable row level security;
alter table leads force row level security;

create policy tenant_isolation on leads
  for all to app_user
  using (tenant_id = current_setting('app.tenant_id', true)::uuid)
  with check (tenant_id = current_setting('app.tenant_id', true)::uuid);
```

Staff visibility (staff see only their own leads and appointments) is enforced in the service layer from the membership role, with an additional policy on `leads`, `appointments`, `conversations` checking `assigned_membership_id` when `app.role = 'staff'`.

Release gates

- A CI test lists every table with a `tenant_id` column and fails if RLS or the policy is missing.
- A cross-tenant test creates two tenants and asserts zero rows leak through API, jobs, exports and webhooks.

- No missing `app.tenant_id` ever returns rows: `current_setting(..., true)` returns null, so the policy matches nothing.

Storage: private buckets; object paths start with `tenant_id/`; files served only through short-lived signed URLs created by the backend.

13. Key indexes and constraints

Every index starts with `tenant_id`; the unique constraints below are what make webhooks, jobs and payments duplicate-safe.

Duplicate-safety constraints

Constraint	Prevents
`webhook_events (provider, provider_event_id)` unique	Same webhook processed twice
`messages (tenant_id, provider_message_id)` unique	Duplicate stored message
`contacts (tenant_id, normalized_phone)` unique	Duplicate contact
`events (tenant_id, dedupe_key)` unique	Same engine event applied twice
`actions (tenant_id, idempotency_key)` unique	Same quote, link, invoice or message sent twice
`payments (tenant_id, razorpay_payment_id)` unique	Double ledger entry
`invoices (tenant_id, number)` unique + sequence row lock	Duplicate or skipped invoice numbers
`meta_events (tenant_id, event_id)` unique	Duplicate conversion events
Exclusion on `appointments` / `slot_holds` per staff time range	Double-booking a staff member
Partial unique: one open lead per contact (`leads (tenant_id, contact_id) where closed_at is null and deleted_at is null`)	Two active leads for one person

Query indexes

- `leads (tenant_id, stage, updated_at desc)`: pipeline board and filters
- `leads (tenant_id, assigned_membership_id, stage)`: staff views
- `leads (tenant_id, band, created_at desc)`: score dashboard
- `leads (tenant_id, origin, ad_referral_id)` and `(tenant_id, chat_link_id)`: source reports
- `messages (tenant_id, conversation_id, created_at desc)`: inbox
- `timeline_events (tenant_id, lead_id, occurred_at desc)`: customer timeline
- `actions (tenant_id, status, next_attempt_at)`: worker pickup and Failed Actions screen
- `appointments (tenant_id, assigned_membership_id, starts_at)`: calendars
- `payments (tenant_id, status, link_expires_at)`: expiry and reminders
- `sync_outbox (status, next_attempt_at)`: sync worker

14. Data retention

These are proposed defaults to confirm with legal and tax advisers; a nightly job applies them per tenant.

Data	Keep for	Then
Message bodies and media	12 months after last activity on the conversation	Body and media removed; metadata kept for reporting
Raw webhook payloads	30 days	Payload removed; row with IDs kept for dedupe
Leads, timeline, quotes, appointments	Life of the subscription	Exported and deleted on tenant closure after a 90-day grace
Invoices, credit notes, payments, refunds	As required by GST record-keeping rules (commonly 72 months; CA to confirm)	Archived read-only; never hard-deleted before the period ends
Audit log	3 years	Deleted
Usage counters	3 years	Aggregated monthly

Data	Keep for	Then
Backups	Daily, kept 7 days (Supabase Pro default) + monthly snapshot kept 12 months	Expire automatically

Deletion requests: a contact's personal data can be erased on verified request; financial records keep the minimum fields the law requires, with the contact's name replaced by a reference.

15. Open questions

- [ ] Can a contact have more than one open lead at a time (e.g. two separate shoots)? The model currently allows one.
- [ ] Should staff log in with phone OTP as well as email?
- [ ] Retention periods in section 14: confirm with legal and CA
- [ ] Store message bodies encrypted at the column level, or rely on database encryption at rest?
- [ ] Multiple workflows per tenant (e.g. separate flows for two service lines) in MVP, or one?
- [ ] Invoice numbering format per tenant (prefix, financial-year reset) confirmed with each client's CA